

Lista 2 — O simulador

Implementar instruções e observar a memória

Microcontroladores — DENE/UFMT · trabalho individual, para casa

Nome

Matri-
cula

OBJETIVOS

- Implementar, no simulador, instruções que operam diretamente sobre a memória.
- Usar o despejo de memória para observar o estado da máquina, na ausência de qualquer forma de impressão.
- Demonstrar experimentalmente que o acumulador não participa dessas operações.
- Comparar o custo de uma tarefa escrita ao estilo registrador-memória e ao estilo *load-store*.

1 Preparação

Baixe `pic18.py`. Não precisa instalar nada além do Python 3.

```
$ python3 pic18.py programa.s --trace 10 --dump
```

Opção	O que faz
<code>--trace N</code>	Mostra as N primeiras instruções executadas, com o custo em ciclos
<code>--dump</code>	Ao final, despeja W , a pilha e os 64 primeiros bytes da memória
<code>--passos N</code>	Interrompe após N instruções. Útil para laços infinitos
<code>--pino D,0</code>	Mede as comutações de RD0 e calcula a frequência

O arquivo de entrada pode ser um `.hex` gravável no chip ou um `.s` em texto, que o próprio simulador monta. O montador aceita rótulos, registradores por nome (`LATD`) ou por número (`0x20`), e os sufixos `,f`, `,w` e número de bit.

```
        clrf    0x20
        movlw   99
laco:    incf    0x20,f
        bra     laco
```

ATENÇÃO

Não há `print`. Este é o ponto da lista, não uma limitação a contornar.

No chip real não existe terminal: para saber o valor de uma variável você acende um LED, envia por serial ou olha a memória pelo depurador. O `--dump` é o equivalente da terceira opção, e é assim que se depura um sistema embarcado.

2 Parte 1 — A instrução que não existe

Rode o programa acima. Ele para na terceira instrução:

```
0004  2A20  ??? 0x2A20 (nao implementada)
```

O montador conhece `incf` e produziu o opcode correto. O **executor** não sabe o que fazer com ele. Sua tarefa é ensiná-lo.

TAREFA**E2.1 — Implementar `incf`.**

- (a) Consulte o formato de `INCF` no conjunto de instruções do datasheet e anote os campos.
- (b) Implemente-a em `executa()`, seguindo o padrão do `decfsz` que já está lá. Respeite o bit *d*.
- (c) Rode o programa e mostre o despejo. Quanto vale `0x20`? Quanto vale `W`?
- (d) Explique o que o valor de `W` demonstra sobre a arquitetura.

3 Parte 2 — Duas mais, e uma sem acumulador

TAREFA**E2.2 — `addwf` e `movff`.**

- (a) Implemente `ADDWF f,d,a`, que soma `W` ao conteúdo de `f`. Consulte o formato no datasheet.
- (b) Implemente `MOVFF fs,fd`. Atenção: ela ocupa **duas palavras** — a segunda contém o endereço de destino. Você precisará ler a palavra seguinte e avançar o contador de programa. Custo: dois ciclos.
- (c) Escreva um programa que coloque 7 em `0x30`, some 5 a ele, e copie o resultado para `0x31`. Mostre o despejo.
- (d) Quantas instruções `MOVFF` economiza em relação a fazer a mesma cópia com `movf` e `movwf`? E quantos ciclos?

4 Parte 3 — Os dois estilos, medidos

TAREFA

E2.3. Some três variáveis, guardadas em `0x40`, `0x41` e `0x42`, deixando o resultado em `0x43`.

- (a) Escreva ao **estilo registrador-memória**, usando `addwf` com destino na memória sempre que possível. Conte instruções e ciclos.
- (b) Escreva ao **estilo load-store**: toda operação passa por `W`, e a memória só é tocada por `movf` e `movwf`. Conte de novo.
- (c) Compare. Explique por que a diferença é menor do que a discussão sobre arquitetura faria supor.
- (d) Agora considere a tarefa oposta: incrementar em uma única variável. Compare os dois estilos nesse caso e explique por que aqui a diferença é grande.

5 Parte 4 — Olhando o despejo

TAREFA

E2.4. Rode este programa com `--passos 12 --dump` e responda **antes** de interpretar o resultado.

```
movlw 0xAA
movwf 0x50
movwf 0x51
```

```
        clrf    0x51
        movlw   0x0F
        addwf   0x50,f
fim:     bra     fim
```

- (a) Preveja o conteúdo de 0x50 e 0x51 ao final. Escreva a previsão antes de executar.
- (b) Execute e compare.
- (c) Nada no programa escreve em LATD. Ainda assim, o que o despejo mostraria se o programa tivesse acionado um pino? Em que região da memória?
- (d) O despejo mostra apenas os 64 primeiros bytes. Modifique `despeja()` para aceitar um endereço inicial e exibir a região dos registradores especiais. Rode o `blink.hex` com essa modificação e localize LATD.

6 Parte 5 — O erro que compila

TAREFA

E2.5. Na sua implementação do `incf`, troque a máscara de 0x2800 para 0x2C00.

- (a) Que instrução 0x2C00 designa? Consulte o datasheet.
- (b) Rode o programa da Parte 1. Descreva o que acontece.
- (c) Este material cometeu exatamente esse tipo de erro ao gerar código à mão: um `decfsz` codificado como 0x2200, que é `ADDWFC`. Descreva o sintoma que isso produziu num programa de piscar LED, e explique por que ele é mais caro de achar do que um erro que impedisse a compilação.
- (d) Que procedimento teria detectado o erro em segundos?

7 Entrega

ENTREGA

Um arquivo compactado contendo:

1. `pic18.py` modificado, com as instruções implementadas.
2. Os arquivos `.s` que você escreveu.
3. Um documento de no máximo **três páginas** com as respostas, incluindo os despejos de memória colados como texto.

Prazo: início da aula seguinte. Trabalho individual — discutir com colegas é bem-vindo, entregar código idêntico não.

CONCEITO

Por que esta lista existe. Vocês contaram ciclos a mão na Aula 2 e vão medi-los no osciloscópio no Roteiro 2. Esta lista é o terceiro método, e o único em que vocês constroem o instrumento em vez de usá-lo.

Depois de escrever `executa()`, a frase “o processador busca, decodifica e executa” deixa de ser uma definição decorada e passa a ser algo que vocês implementaram. É a diferença entre saber o que uma instrução faz e saber por que ela faz.